

AFRILANG Stdlib

std/c/statmape

Bibliothèque AFRILANG `std/c/statmape` (niveau complex, catégorie complex). Elle expose 6 fonction(s) :
mape, wape, smap

```
`import "std/c/statmape"` · `use statmape`
```

- `mape(actual list number, predicted list number) ? number`
- `wape(actual list number, predicted list number) ? number`
- `smap(actual list number, predicted list number) ? number`
- `bias(actual list number, predicted list number) ? number`
- `trackingSignal(actual list number, predicted list number) ? number`
- `forecastError(actual list number, predicted list number) ? list number`

Category: complex | Tier: complex | Functions: 6

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/statmape` (niveau complex, catégorie complex). Elle expose 6 fonction(s) : mape, wape, smape

```
`import "std/c/statmape"` . `use statmape`  
- `mape(actual list number, predicted list number) ? number`  
- `wape(actual list number, predicted list number) ? number`  
- `smape(actual list number, predicted list number) ? number`  
- `bias(actual list number, predicted list number) ? number`  
- `trackingSignal(actual list number, predicted list number) ? number`  
- `forecastError(actual list number, predicted list number) ? list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statmape"  
use statmape
```

Niveau : complex · Catégorie : Modules complex (c/)

Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? mape(actual list number, predicted list number) ? number  
? wape(actual list number, predicted list number) ? number  
? smape(actual list number, predicted list number) ? number  
? bias(actual list number, predicted list number) ? number  
? trackingSignal(actual list number, predicted list number) ? number  
? forecastError(actual list number, predicted list number) ? list
```

4. Exemples d'utilisation

```
import "std/c/statmape"  
use statmape  
  
create result = mape(/* args */)   
say result  
  
create result = wape(/* args */)   
say result  
  
create result = smape(/* args */)   
say result  
  
create result = bias(/* args */)   
say result  
  
create result = trackingSignal(/* args */)   
say result  
  
create result = forecastError(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/statmape"` en tête de fichier.  
? Lancez `afirilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module statmape  
  
function mape(actual list number, predicted list number) returns number  
end
```

```
function wape(actual list number, predicted list number) returns number
end

function smape(actual list number, predicted list number) returns number
end

function bias(actual list number, predicted list number) returns number
end

function trackingSignal(actual list number, predicted list number) returns number
end

function forecastError(actual list number, predicted list number) returns list number
end
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/statmape` (tier complex, category complex). Exports 6 function(s): mape, wape, smape, bias, trac

```
`import "std/c/statmape"` . `use statmape`  
- `mape(actual list number, predicted list number) ? number`  
- `wape(actual list number, predicted list number) ? number`  
- `smape(actual list number, predicted list number) ? number`  
- `bias(actual list number, predicted list number) ? number`  
- `trackingSignal(actual list number, predicted list number) ? number`  
- `forecastError(actual list number, predicted list number) ? list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/statmape"  
use statmape
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? mape(actual list number, predicted list number) ? number  
? wape(actual list number, predicted list number) ? number  
? smape(actual list number, predicted list number) ? number  
? bias(actual list number, predicted list number) ? number  
? trackingSignal(actual list number, predicted list number) ? number  
? forecastError(actual list number, predicted list number) ? list
```

4. Usage examples

```
import "std/c/statmape"  
use statmape  
  
create result = mape(/* args */)   
say result  
  
create result = wape(/* args */)   
say result  
  
create result = smape(/* args */)   
say result  
  
create result = bias(/* args */)   
say result  
  
create result = trackingSignal(/* args */)   
say result  
  
create result = forecastError(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/statmape"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module statmape  
  
function mape(actual list number, predicted list number) returns number  
end
```

```
function wape(actual list number, predicted list number) returns number
end

function smape(actual list number, predicted list number) returns number
end

function bias(actual list number, predicted list number) returns number
end

function trackingSignal(actual list number, predicted list number) returns number
end

function forecastError(actual list number, predicted list number) returns list number
end
end
```